[redacted]

（装订线内不要答题）

复旦大学计算机科学技术学院

《数据结构》期中考试试卷

共7页

[redacted]

11 月

6 ~~6~~

题号	一	二	三	总分
得分	17	29	23	

17

一、填空题（本大题共十个填空，每空 2 分，共 20 分）

1、 时间复杂度 $O(n\log_2 n+5000n)$与 $O(n^{\log_2 n}-6000n)$相比，复杂度较低（即允许速度较快）的是 $O(n\log_2 n+5000n)$

2、 已知数组 A[1..10, 1..10]为对称矩阵，其中每个元素占 5 个单元。现将其下三角部分按行优先次序存储在起始地址为 1000 的连续内存单元中，则地址 1095 中存放的元素在数组 A 中的位置是__________。 A[6][5]

3、 带头结点的循环链表，first 为指向表头结点的指针，每个结点中由 link 指针指向下一个结点，用 C 语言语法表达来判断该链表只有一个元素结点（不包括头结点）的条件为 first→link→link == first

4、 后缀表达式 xy+10/ab*c/+xy-ab-*-对应的中缀表达式为 ((x+y)/10 + a*b/c)-(x-y)*(a-b)

5、 广义表 GL=((a, b, c),(d, e, f))，假设求表头操作为 head，求表尾操作为 tail，则 head[tail[tail[head[tail[GL]]]]]= f 。

6、 已知完全二叉树的第 7 层有 10 个叶结点，规定根结点层次为 1，则整棵二叉树的结点数最多为 235 。

7、 将一个有 n 个结点的森林用左孩子-右兄弟链表示时，其中空链域的个数为 n+1 。

8、 给定具有 n 个结点的有序树先根遍历序列，这棵树可能的形态有_______种。

$\frac{(2n)!}{n!}$ ✗ $\cdot \frac{1}{n+1}$

9、 将关键码序列{16, 53, 23, 94, 31, 72}转换为最大堆，所形成的二叉树最后一个叶结点的值为 23 。

10、n 个元素形成的堆，向其中插入一个新元素后，重新调整成堆的时间复杂度是 $O(\log_2 n)$

6 ~~6~~

二、问答题（本大题共六小题，共 40 分）

1、某算法时间复杂度的递推关系如下，其中 n 为求解问题的规模，设 n 是 3 的正整数幂。请给出该算法时间复杂度的大 O 表示法及推导过程。（6 分）

$$T(n)=\begin{cases}1 & (n=1)\\ 5T(n/3)+n & (n>1)\end{cases}$$

解：$T_n + \frac{3}{2}n = 5\left(T(n/3) + \frac{3}{2}\cdot\frac{1}{3}n\right)$

∴ $T_n + \frac{3}{2}n = 5^{\log_3 n}\cdot\left(T(1) + \frac{3}{2}\right)$

$T_n = O(5^{\log_3 n})$

×

ink == first

-y) *(a-b)

3

2、 已知目标串 T="aaaaaacaaaaaab"，模式串 P="aaaaaab"。（7 分）

a) 请按照 KMP 算法的要求计算模式串 P 的 Next[ ]数组，计算用 KMP 算法根据 Next 数组在 T 上进行匹配所需要的字符比较次数。

b) 仍然按照从左向右的匹配顺序，与针对 a)回答中所进行 Next[ ]数组计算相比，是否还有更优的计算方式，请描述具体计算思路。

a) Next[0]=-1 Next[1]=0 Next[2]=1 Next[3]=2

Next[4]=3 Next[5]=4 Next[6]=5

T: aaaaaa c aaaaaab　　T: aaaaaa c

p aaaaaa b　　　　　　aaaaaa b

比较7次　　　　　　　比一次

T: aaaaaa c aaaaaab　比7次

aaaaaab

……

共比 21 次

b).

前

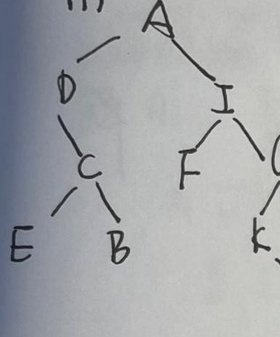
3、 森林的后根次序遍历结果为 DECBAFIKLG，先根次序遍历结果为 ADCEBIFGKL。要求：

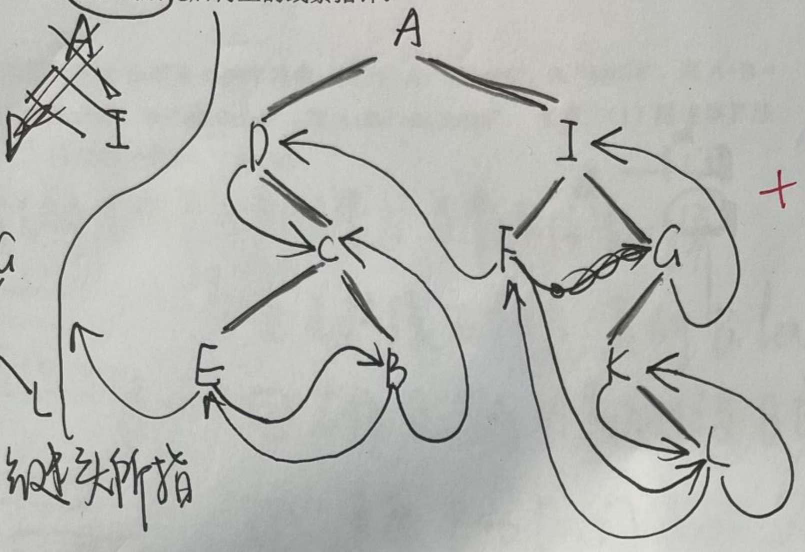
(1) 画出这个森林对应的二叉树；

(2) 画出这棵二叉树经过后序线索化后树上的线索指针。

（6分）

(1)





+6

(2) 如图中箭头所指

4、 已知指向二叉树中某结点的指针 p，并且已知该二叉树为中序线索化二叉树，规定其中的中序第一个结点的左孩子指针为空，最后一个结点的右孩子指针为空。请问仅根据这些已知条件什么情况下能找到 p 的双亲结点，什么情况下无法找到？（7分）

+3

~~对中序第一~~个

常规思路 调用 last 函数 得 p 左子树中最后访问

若p为左孩子，只要找到~~右~~子树中最后访问结点，

它的后继（右孩子）就是p的父亲；

若是上层结点，无法找到

[illegible]

求：

+6

定其中的中
这些已知条

+7

访问

后访问结点，

亲；

5、已知某完全 k 叉树只有度为 k 的结点及叶结点，设叶结点数为 n_0，求树的高度 h。（7 分）

对于满k叉树，有m层 → 共 k^m-1 个结点

对于 n_0，即要找到 ~~m₁、m₂~~ m' 满足

$k^{m'}-1 \le n_0 \le k^{m'+1}-1$

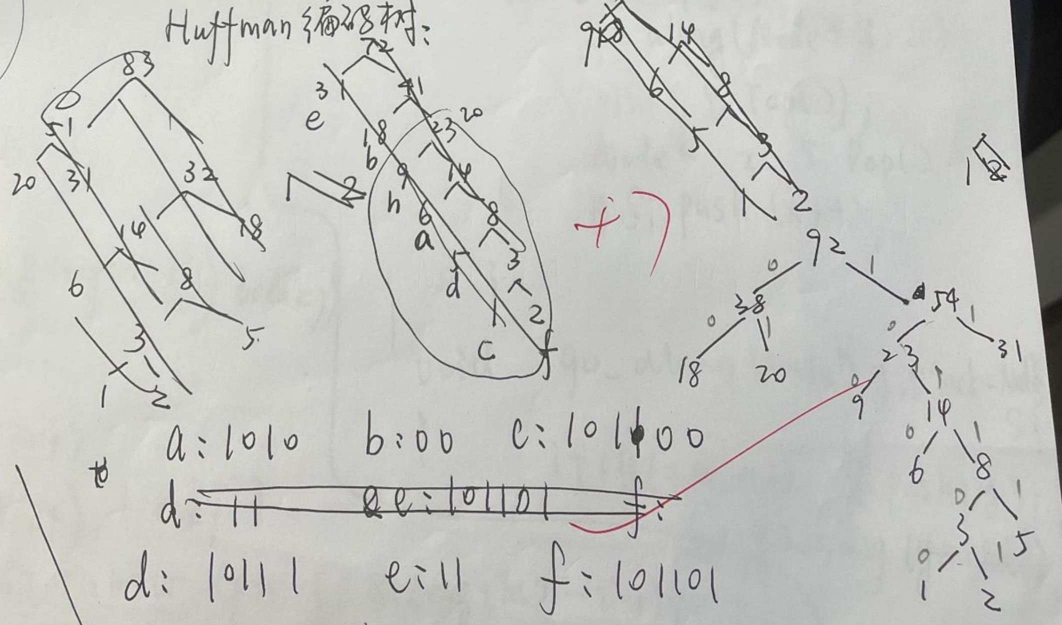
此时 $m'+1$ 即为结果

+4

∴ $h=m'+1=\lceil \log_k(n_0+1) \rceil$

6、用于通信的电文由 8 个字符组成{a, b, c, d, e, f, g, h}，字符在电文中出现的频率分别为 6, 18, 1, 5, 31, 2, 20, 9。试为这 8 个字符设计哈夫曼编码。使用 0~7 的 3 位等长二进制表示形式是另一种编码方案，对于上述实例比较两种方案的优缺点。（7 分）

Huffman编码树：



a: 1010　b: 00　c: 101100

~~d: 11　e: 101101　f:~~

d: 10111　e: 11　f: 101101

g: 01　h: 100

总需空间 4×6+18×2+1×6+5×5+31×2+2×6+20×2+9×3=232

而采用等长需 92×3=276.

∴ Huffman编码更省空间

但二进制表示更均匀，长短整齐划一

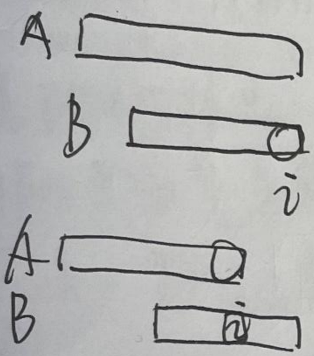
7

三、算法设计题（本大题共四小题，共 40 分）

（以下题目，首先用简明的文字写出算法基本思路，然后给出数据结构和算法的 C 或 C++描述。）

1、用尽量快的算法实现连接两个头尾重合的字符串。例如：A: “123456”，B: “45678”，则 A+B = “12345678”，再如 A: “abcdef”，B: “[illegible]cdefgh”，则 A+B=“abcdefgh”。 要求：（1）描述该算法的基本设计思想；（2）给出程序代码。（10 分）

解 (1)：首先将 B的最后一位置与 A最后位置对齐



①先持续右移B，直到 B.lengt

②右移 B，直到 A最后一位与 B[i]

③. 考察 B的[0,i]是否与A后i

④ 相同，将 A ~~+~~ += B.subs

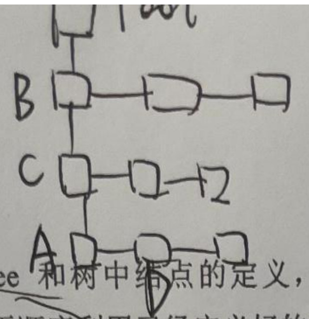
不同：重复②、③

(2)

```
~~find~~ void add (string s1, string s2){
    int pos = min (B.length()-1, A
    ~~for (int while (B[pos] != A~~
    while(1){
        ~~for~~ while(pos >= 0) { if (B[po
                                        pos--
            int i=0;
            for (int i=0, i !=
```

③

设计思路

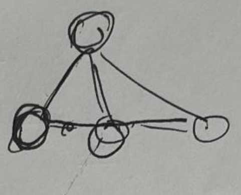


child

先将B、C、A入栈
当A没有孩子时，访问A
然后将A入栈，[illegible]

2、给定以下的 m 次树 Tree 和树中结点的定义，请给出 Tree 上的非递归后根遍历算法 s_postOrder( )，以后根遍历顺序利用已经定义好的函数 visit(**struct** Node *n)访问每个结点。假设已经有基本数据结构栈和队列的定义，算法中可以类似于教材中的方式使用。（15 分）

```
struct Node {
        DataType data;
        struct Node *child[m];
    };
    class Tree {
    {
          public:
                struct Node *root;
                void s_postOrder(struct Node *t);
    };
```



```
void Tree::s_postOrder( struct Node
                                    t

stack<Node*>  S;
S.push(t);  Node* y;
~~While Node*  y = S.top()~~;
        ~~S.pop()~~;
while (!S.empty())  {  y = S.
    go_along(Node* y, S)
    visit( S.Top());
    Node*   x = S.Pop();
    S.push (x++);
}
```

th();

访问

length()-(i+1))

)-1);

];length()-1]) break;

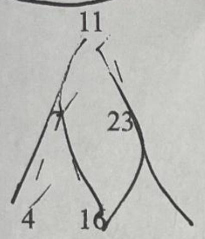
}

}

void go_along(Node*x,

3、输入一个整数和一棵二叉树。从树的根结点开始往下访问一直到叶结点所经过的所有结点形成一条路径。请编写算法打印出和与输入整数相等的所有路径。假设已经有基本数据结构栈和队列的定义，算法中可以类似于教材中的方式使用。

例如，输入整数 34 和如下二叉树：



则打印出两条路径：11, 23 和 11, 7, 16。

+13

要求：(1) 描述该算法的基本设计思想；(2) 给出程序代码。（15 分）

(1) 若已有根结点 root，以及目标值 target，
那么只需要考虑在左右子树中路径是否有使值为 target —
下设一打印函数，顺序输出队列的值；递归查

不相同

直

(2)
```
void print (list<int> q) {
  list<int> :: iterator  iter = q.begin();
    while ((iter+1) != q.end()) {
      printf ("%d,", *iter);
      iter ++;
  }
    printf ("%d", *iter);
}
void
  find-way (TreeNede<int>* root, list<int> & q
```